SQL Basics

1) What SQL is

SQL stands for **Structured Query Language**. It is used to create databases and tables, insert and update data, retrieve information, and organize records in a way that applications and analysts can use efficiently.

SQL is not a general-purpose programming language like Python or C++; it is a database language designed for working with structured data.

2) SQL environment

To learn SQL well, you should work with a database system such as MySQL, PostgreSQL, SQL Server, or SQLite. Most beginners start by creating a small local database and practicing queries on sample tables.

A common workflow is:

1. Install a database engine.
2. Create a database.
3. Create tables.
4. Insert sample data.
5. Query the data.
6. Modify and analyze it.

3) Core SQL commands

The foundation of SQL is the CRUD pattern:

- CREATE for databases and tables.
- INSERT for adding rows.
- SELECT for reading data.
- UPDATE for changing data.
- DELETE for removing data.

Example:

sql

CREATE DATABASE school;

USE school;

CREATE TABLE students (

```
    id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT,
    marks INT
);

INSERT INTO students (id, name, age, marks)
VALUES
(1, 'Asha', 20, 91),
(2, 'Rahul', 21, 84),
(3, 'Priya', 19, 95);
```

4) Querying data

The most used SQL statement is SELECT. You use it to retrieve columns from a table, and then add filters, sorting, grouping, and limits.

Example:

sql

```
SELECT name, marks
FROM students
WHERE marks > 85
ORDER BY marks DESC;
```

Useful clauses:

- WHERE filters rows.
- ORDER BY sorts rows.
- DISTINCT removes duplicates.
- LIMIT returns a fixed number of rows.

5) Filtering and operators

SQL filtering uses comparison and logical operators. The most important ones are:

- =, !=, >, <, >=, <=
- AND, OR, NOT
- IN, BETWEEN, LIKE

Example:

sql

SELECT *

FROM students

WHERE marks BETWEEN 80 AND 95

AND name LIKE 'P%';

6) Updating and deleting

UPDATE changes existing rows, and DELETE removes rows. These commands should always be used carefully, especially with WHERE, because without it you may affect the entire table.

Example:

sql

UPDATE students

SET marks = 97

WHERE id = 3;

DELETE FROM students

WHERE id = 2;

7) Table design basics

A good database starts with good design. Tables should usually represent one entity, such as students, orders, products, or employees. Each table should have a primary key, and related tables should use foreign keys to connect data.

Important concepts:

- Primary key: uniquely identifies a row.
- Foreign key: links one table to another.
- Not null: disallows empty values.

- Unique: prevents duplicate values.
- Check: enforces rules.
- Default: sets a default value.

8) Joins

Joins combine data from multiple tables. This is one of the most important topics in SQL and usually comes after the basics.

Common joins:

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL JOIN

Example:

sql

```
SELECT students.name, courses.course_name
FROM students
INNER JOIN enrollments ON students.id = enrollments.student_id
INNER JOIN courses ON enrollments.course_id = courses.id;
```

9) Aggregation and grouping

SQL is very powerful for analysis because it can summarize data using aggregate functions. The main ones are COUNT, SUM, AVG, MIN, and MAX.

Example:

sql

```
SELECT COUNT(*) AS total_students,
    AVG(marks) AS average_marks
FROM students;
```

If you want results by category, use GROUP BY:

sql

```
SELECT age, COUNT(*) AS total
FROM students
GROUP BY age;
```

10) Subqueries and views

A subquery is a query inside another query. Views are saved queries that behave like virtual tables. These are common in intermediate and advanced SQL.

Example subquery:

sql

```
SELECT name
FROM students
WHERE marks = (SELECT MAX(marks) FROM students);
```

Example view:

sql

```
CREATE VIEW top_students AS
SELECT name, marks
FROM students
WHERE marks >= 90;
```

11) Functions and advanced SQL

A full SQL course should also cover string functions, date functions, conditional logic, and window functions. These are especially important for data analysis and reporting.

Examples:

sql

```
SELECT UPPER(name) FROM students;

SELECT CURRENT_DATE;

SELECT CASE
    WHEN marks >= 90 THEN 'A'
    WHEN marks >= 75 THEN 'B'
    ELSE 'C'
END AS grade
FROM students;
```

12) Practice roadmap

A good learning path is:

1. SQL basics.
2. Table creation.
3. Insert/update/delete.
4. Filtering and sorting.
5. Joins.
6. Aggregation.
7. Subqueries.
8. Views and indexes.
9. Stored procedures and triggers.
10. Mini projects.

This is close to how many full SQL courses are structured, from beginner topics to advanced database work.

13) Small project: Student database

Here is a simple project you can build and practice with.

Project goal

Create a student database that stores students, courses, and enrollments, then query marks, top performers, and course-wise summaries.

Tables

sql

```
CREATE DATABASE college;
USE college;

CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT,
    city VARCHAR(50)
);

CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL
);
CREATE TABLE enrollments (
    enrollment_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    marks INT,
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

Insert sample data

sql

```
INSERT INTO students VALUES
(1, 'Asha', 20, 'Pune'),
(2, 'Rahul', 21, 'Mumbai'),
(3, 'Priya', 19, 'Delhi');

INSERT INTO courses VALUES
(101, 'SQL Basics'),
(102, 'Data Analysis'),
(103, 'Database Design');

INSERT INTO enrollments VALUES
(1, 1, 101, 92),
(2, 1, 102, 88),
(3, 2, 101, 76),
(4, 3, 103, 95);
```

Useful queries

sql

```
SELECT * FROM students;

SELECT name, city
FROM students
WHERE city = 'Pune';

SELECT s.name, c.course_name, e.marks
FROM enrollments e
JOIN students s ON e.student_id = s.student_id
JOIN courses c ON e.course_id = c.course_id;
```

```
SELECT c.course_name, AVG(e.marks) AS avg_marks
FROM enrollments e
JOIN courses c ON e.course_id = c.course_id
GROUP BY c.course_name;
```

```
SELECT s.name, e.marks
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
WHERE e.marks = (SELECT MAX(marks) FROM enrollments);
```

14) What to learn next

After SQL basics, the most valuable next topics are:

- Database normalization.
- Indexes.
- Query optimization.
- Transactions and ACID.
- Stored procedures.
- Triggers.
- Window functions.
- SQL for analytics.

15) Final study plan

If you want to learn SQL like a course, follow this sequence:

- Week 1: basics and CRUD.
- Week 2: filtering, joins, and grouping.
- Week 3: subqueries, views, and functions.
- Week 4: normalization, indexes, transactions, and project work.